Politecnico di Torino

Master's Degree in Computer Engineering

Migrating Legacy Services to a Cloud-Native Architecture: A Case Study on Outsight Cloud

outsight

Academic Supervisors: Prof. Fulvio Giovanni Risso, Prof. Adlen Ksentini
Company Supervisors: Kasper Rutten, Anatolii Kostrygin

Candidate: Giovanni Russo
Matricola: s343424

Academic Year 2025–2026

Abstract

Legacy systems often evolve through years of incremental development, resulting in architectures that become difficult to scale, maintain, and modernize. This thesis investigates the migration of Outsight Cloud, a production platform for managing LiDAR-related workflows and assets, from a legacy service-oriented architecture toward a cloud-native environment on AWS.

Initially, the platform was deployed on virtual machines provisioned on the DigitalOcean cloud and orchestrated through Docker Compose, which managed the lifecycle of containerized application services directly on each VM without a dedicated container orchestration platform. Core business entities were stored in Airtable, which served as the primary operational database, while additional external services supported storage, documentation, and notification workflows. Over time, this architecture progressively introduced scalability, consistency, maintenance, and operational-governance challenges.

The thesis proposes and evaluates an incremental migration strategy based on Infrastructure as Code, managed cloud services, and progressive coexistence between legacy and target components. Particular attention is given to the transition from Airtable to PostgreSQL, the migration from VM-based deployments to managed container services, the adoption of Terraform-driven infrastructure provisioning, and the introduction of validation mechanisms designed to reduce migration risk while preserving service continuity. The proposed approach combines architectural decision-making with technical and operational evaluation criteria, including reproducibility, maintainability, deployment reliability, observability, and cost sustainability. To reduce migration risk and preserve system continuity, the project adopts incremental validation practices, automated consistency checks, and reproducible deployment workflows across isolated environments. The resulting architecture demonstrates

how cloud-native modernization can be achieved incrementally in a real industrial context through controlled evolution, continuous validation, and risk-aware design practices. Beyond the specific case study, the migration principles and engineering practices discussed throughout the thesis are applicable to broader legacy modernization scenarios, offering transferable insights for organizations facing similar cloud adoption challenges.

Acknowledgements

I would like to express my sincere gratitude to Outsight for providing me with the opportunity to complete this internship and work on challenging and meaningful projects. I am particularly thankful to my supervisor and the entire development team for their guidance, support, and patience throughout this journey.

I would also like to thank the Politecnico di Torino for the academic foundation that made this work possible, and my thesis advisor for valuable feedback and encouragement.

Finally, I am deeply grateful to my family and friends for their unwavering support during my studies.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Industrial Context and Legacy Constraints

In recent years, cloud computing has significantly transformed the way software systems are designed, deployed, and maintained. Modern cloud platforms provide scalable infrastructure, managed services, automated deployment capabilities, and operational flexibility that are difficult to achieve with traditional self-managed environments. As a consequence, many organizations are progressively modernizing legacy applications to adopt cloud-native architectures.

However, migrating an existing production platform to the cloud is rarely a straightforward process. Legacy systems are typically the result of years of incremental development shaped by evolving business requirements, operational constraints, and technological decisions. Over time, these systems often accumulate architectural dependencies and operational practices that complicate scalability, maintainability, and long-term evolution.

This thesis analyzes the migration of Outsight Cloud (OC), a platform used to manage LiDAR-related workflows, simulations, organizations, sites, and associated resources. Before the migration effort, the platform relied on a virtual-machine-based infrastructure hosted on DigitalOcean Droplets and orchestrated through Docker Compose, integrating several external services, most notably Airtable for core data management. This architecture enabled rapid development and operational flexibility during the early stages of the product lifecycle, allowing new features to be delivered with limited infrastructure overhead. As platform usage and

integrations expanded, however, this approach began to show signs of architectural strain.

1.2 Problem Statement

The engineering problem addressed in this work concerns the migration of Outsight Cloud from a legacy architecture toward a cloud-native environment under real industrial and operational constraints.

Over time, the platform became strongly dependent on external services and platform-specific integrations, especially Airtable, which was deeply embedded in application workflows and operational processes. The absence of a fully relational data model limited consistency guarantees and complicated the management of relationships between entities. Operational procedures increasingly relied on manual or partially automated workflows, generating inefficiencies and increasing the risk of human error. In parallel, the original deployment model became progressively harder to scale and maintain as infrastructure requirements and operational complexity continued to grow.

To address these limitations, the company initiated a cloudification process aimed at redesigning the platform architecture and migrating toward a managed cloud environment. The selection of the target platform and architectural strategy was the result of a structured evaluation of candidate solutions against technical, operational, and economic criteria. The main engineering objective was to regain explicit architectural ownership over core data and infrastructure assets through Infrastructure as Code practices, the migration from Airtable to PostgreSQL, the introduction of managed cloud services, and the modernization of deployment and operational workflows. A central challenge was defining an execution sequence capable of reducing technical and operational risk before final cutover, relying on validation workflows, reproducible infrastructure management, and controlled transition mechanisms.

Beyond addressing these architectural limitations, the migration initiative was driven by a set of concrete business and engineering objectives:

- Reduce operational costs by replacing Airtable and self-managed infrastructure with managed cloud services.

- Improve system performance and data access efficiency through the adoption of PostgreSQL as the primary relational datastore.
- Reduce operational overhead by automating infrastructure provisioning, deployment, and maintenance through Infrastructure as Code practices.
- Re-establish ownership of core data models and infrastructure definitions, reducing dependency on vendor-specific platforms.
- Minimize migration risk and service disruption through incremental coexistence, validation workflows, and controlled transition mechanisms.
- Establish a scalable and maintainable architectural foundation capable of supporting long-term platform evolution.

1.3 Thesis Structure

This thesis is organized as follows:

Chapter 2 introduces the conceptual and related-work foundations on cloud-native modernization, migration patterns, and IAM evolution in legacy systems.

Chapter 3 describes the case-study baseline (as-is architecture), including technical debt and cost-related migration drivers.

Chapter 4 presents the migration methodology and decision framework, including candidate options and selection rationale.

Chapter 5 defines the target architecture (to-be), its main components, and design trade-offs.

Chapter 6 details the incremental migration implementation foundations, with focus on IaC, data migration tooling, and validation pipeline.

Chapter 7 discusses the evaluation approach and current evidence regarding migration feasibility, quality, and impact.

Chapter 8 concludes the thesis by summarizing contributions and outlining future work.

Chapter 2

Background and Related Work

This chapter provides the conceptual background required to frame the migration problem and positions the thesis with respect to related work.

2.1 Cloud-Native Principles for Legacy Modernization

Cloud-native modernization is not limited to container adoption or workload relocation. In migration contexts, it also requires reproducible infrastructure, explicit service boundaries, deployment automation, operational observability, and validation mechanisms capable of supporting incremental system evolution. These principles become particularly important when architectural changes affect production workflows, persistent data, and operational procedures.

In *Building Microservices*, Newman frames modern service-oriented architectures around independently deployable services with explicit boundaries and hidden internal state [1]. This principle is relevant to legacy modernization because it separates external contracts from internal implementation details, reducing the risk that changes in one component propagate unexpectedly across the platform.

Modern cloud-native systems are typically designed around automation-oriented practices and managed services that simplify infrastructure operations and improve scalability. However, migrating a legacy platform toward such architectures requires careful consideration of production constraints, compatibility requirements,

and long-term maintainability.

2.1.1 Incremental Delivery and Operability

A recurring principle in modernization literature is the preference for incremental delivery over full-system replacement. Incremental migration reduces operational risk, enables earlier feedback cycles, and allows validation activities to be performed progressively throughout the transition process.

Newman argues against big-bang rewrites, emphasizing that large simultaneous replacements tend to concentrate risk and delay feedback [1]. Decomposing migration work into smaller steps makes it possible to start from lower-risk components, build confidence progressively, and validate architectural changes before irreversible decisions are made.

To be effective, incremental delivery requires infrastructure automation, environment consistency, reproducible deployment workflows, and testable interfaces. In industrial systems, these practices make architectural changes observable, reversible, and easier to validate before they affect critical workflows.

2.1.2 Infrastructure as Code

Infrastructure as Code (IaC) is a key enabler for cloud-native modernization because it transforms infrastructure configuration into versioned and reproducible artifacts. Instead of relying on manual provisioning procedures, infrastructure resources are defined declaratively and managed through automated workflows.

Newman identifies Infrastructure as Code as a core deployment principle because infrastructure configuration stored in source control allows environments to be recreated deterministically and shared across teams [1]. In *Infrastructure as Code: Dynamic Systems for the Cloud Age*, Morris further identifies three core practices for effective IaC adoption: defining everything as code, continuously testing and delivering all work in progress, and building small loosely coupled pieces that can be changed independently [2]. Together, these practices improve traceability, reproducibility, reviewability, and consistency across development and production environments. In migration projects, IaC also simplifies environment replication,

deployment governance, and rollback management, reducing operational risks during infrastructure evolution.

2.2 Migration Strategies and Patterns

Cloud migration projects can follow different strategies depending on system complexity, operational requirements, and modernization goals. Common approaches include rehosting, replatforming, refactoring, and selective rebuilding. In practice, enterprise migration projects often combine multiple strategies across subsystems according to their coupling level, criticality, migration effort, and risk exposure.

2.2.1 Cloud Migration Fundamentals

Cloud migration is the process of moving applications, data, and infrastructure resources from traditional environments to cloud-based platforms [3]. In practice, this transition is executed progressively through staged rollout strategies and validation activities designed to minimize disruption.

Historically, many organizations operated software systems on self-managed infrastructures hosted in local data centers or virtualized environments. While these architectures provided direct control over resources, they also introduced significant operational complexity related to infrastructure maintenance, capacity planning, backup management, monitoring, and service availability.

For this reason, cloud migration planning must evaluate not only where workloads will run, but also how infrastructure responsibilities, automation practices, reliability requirements, and operational governance will change in the target environment.

Reference guidance highlights several recurring migration drivers and benefits [3], including cost optimization, elastic scalability, managed operational services, improved infrastructure reliability, and access to modern distributed delivery models.

Cloud migration strategies are generally classified into three major categories [3]:

- **Rehosting**, which moves workloads to the cloud with minimal architectural changes.

- **Replatforming**, which introduces selective adaptations to better exploit cloud capabilities while preserving the overall application architecture.
- **Refactoring**, which redesigns parts of the system to align with cloud-native principles and improve long-term scalability and maintainability.

Migration programs are often organized into assessment, preparation, and migration phases [3]. These phases support technical discovery, environment preparation, pilot execution, and incremental rollout activities.

In practice, real-world migration projects often combine multiple strategies across different system components. Newman also stresses that decomposition and modernization choices should be driven by the desired outcome rather than by technical convenience alone [1]. The Outsight Cloud migration can be primarily classified as a replatforming effort complemented by selective refactoring activities.

The migration preserved the core application architecture and business functionality while replacing key infrastructure and platform dependencies. Examples of replatforming include the transition from DigitalOcean-hosted workloads to AWS managed services and the adoption of ECS Fargate as the runtime environment.

At the same time, selected components required refactoring to support the migration objectives. These changes included the transition from Airtable to PostgreSQL, the introduction of Infrastructure as Code practices through Terraform, and adaptations to deployment and routing workflows required by the target cloud-native architecture.

Consequently, the Outsight Cloud migration is best described as a hybrid strategy in which runtime modernization, data-layer redesign, and infrastructure automation addressed different limitations of the legacy platform.

2.2.2 System Coexistence in Incremental Migration

For production systems that cannot tolerate service interruption, coexistence between legacy and target components becomes a critical architectural requirement. During migration phases, both systems frequently need to operate simultaneously while compatibility and behavioral consistency are progressively validated.

Newman describes the *strangler fig pattern* as a technique for wrapping legacy

systems incrementally and redirecting calls to new implementations without replacing the entire system at once [1]. He also discusses parallel-run approaches, where old and new implementations execute side by side and their results are compared before final cutover [1].

Morris further describes the *expand and contract* pattern as a structured technique for changing live infrastructure without disrupting running services. The pattern involves first adding new resources alongside existing ones, migrating consumers incrementally, and only then removing the legacy components [2]. This approach avoids the risks of big-bang replacements and keeps the system in a consistent, testable state throughout the transition.

This coexistence model introduces additional engineering complexity, including contract compatibility, dual-backend validation, synchronization workflows, rollback capabilities, and progressive switchover mechanisms. As a consequence, migration validation becomes an integral part of the system architecture rather than a secondary operational activity.

2.2.3 Decision-Driven Migration

Related work frequently describes migration patterns from a purely technological perspective, focusing primarily on target architectures and adopted cloud services. In industrial contexts, however, migration decisions must also consider operational constraints, organizational impact, recurring costs, maintainability, and migration risk.

For this reason, migration strategies should be supported by explicit decision frameworks capable of combining technical and economic rationale. This aspect is particularly relevant in large-scale modernization projects where architectural choices affect both infrastructure sustainability and long-term operational governance.

2.3 IAM Modernization in Legacy Systems

Identity and Access Management (IAM) modernization is a recurrent challenge in legacy platforms. Historically, many systems implemented authentication and authorization logic directly inside application workflows and persistence layers, generating tightly coupled identity-management models that become difficult to evolve and maintain over time.

In AWS-based environments, *AWS Identity and Access Management* (IAM) is the web service that governs access to cloud resources: it defines who is authenticated and who is authorized to use them, and associates *principals* (such as IAM users, roles, federated identities, or applications) with permission policies that determine which operations are allowed on account resources [4]. Access is granted only after an authorization request succeeds against the policies in effect for that principal and target service [4].

Modern cloud-native architectures instead rely on standardized protocols and dedicated identity services to improve interoperability, scalability, and security.

2.3.1 From Application-Coupled Identity to Standard Protocols

Legacy systems frequently embed authentication assumptions directly into business logic and data models. Migrating toward standards-based IAM therefore requires decoupling identity management from application-specific implementations and redefining authorization boundaries.

Protocols such as OAuth 2.0 and OpenID Connect (OIDC) provide stronger interoperability and security guarantees than ad-hoc authentication approaches. However, introducing standardized IAM mechanisms into existing systems also requires compatibility validation with existing workflows, clients, and operational procedures.

2.3.2 Open-Source and Managed IAM Trade-offs

The choice between open-source IAM solutions and managed identity platforms affects operational complexity, integration flexibility, infrastructure ownership, and long-term maintenance costs.

Managed IAM services simplify operational governance by reducing infrastructure-management responsibilities and providing built-in support for authentication workflows, user lifecycle management, and security integration. Open-source solutions, on the other hand, generally provide greater customization flexibility and infrastructure control at the cost of increased operational responsibility.

In migration projects, these trade-offs also influence how progressively existing services can adopt modern authentication flows while preserving backward compatibility.

2.4 Limitations in Existing Approaches

The literature provides strong conceptual foundations on cloud migration patterns and IAM technologies. However, industrial guidance often remains fragmented across separate technical dimensions.

2.4.1 Fragmented Treatment of Technical and Economic Criteria

Technical architecture decisions are frequently discussed without detailed consideration of operational costs, infrastructure governance, or long-term maintenance overhead, even though sustainable migration depends on aligning technical choices with those same constraints.

As a consequence, migration projects require evaluation frameworks capable of combining scalability, maintainability, operational sustainability, and economic impact within a unified decision process.

2.4.2 Limited Evidence on Incremental Validation

Many migration case studies focus primarily on final target architectures while providing limited evidence regarding coexistence management and intermediate validation mechanisms.

In industrial environments migration reliability often depends on validation practices such as dual-backend testing, data-consistency verification, environment-separated deployment workflows, and incremental rollout strategies. Morris emphasizes that

continuously testing infrastructure changes as they are developed rather than waiting until the system is complete is essential for catching errors early and reducing the cost of correction [2]. These aspects are essential for reducing migration risk during long transition phases.

2.5 Positioning of This Thesis

This thesis addresses the above limitations through a decision-oriented migration case study focused on incremental architectural evolution under real industrial constraints.

The contribution of the work is not limited to the description of a target AWS architecture. Instead, it analyzes the migration of a production platform from a legacy DigitalOcean-based environment toward a cloud-native architecture built on managed AWS services. The thesis also emphasizes the reasoning process that connects baseline system limitations, migration decisions, infrastructure automation, data-layer transition planning, and validation workflows.

Particular attention is given to Infrastructure as Code adoption, migration reproducibility, environment separation, and validation-oriented migration practices designed to reduce technical and operational risk.

Although the work is centered on Outsight Cloud, the migration principles and engineering practices discussed throughout the thesis are applicable to broader legacy modernization scenarios involving progressive cloud-native adoption.

Chapter 3

Case Study Baseline (As-Is System)

This chapter describes the initial state of the Outsight Cloud platform before the migration strategy was formalized. The objective is to establish the architectural and operational baseline against which migration decisions and modernization efforts are evaluated.

3.1 Current Architecture and Data Flow

Before the migration to AWS, Outsight Cloud relied on a containerized architecture designed to support rapid feature delivery and operational flexibility during the early stages of product development.

The platform managed LiDAR-related workflows and assets through a web application and backend API stack composed of several interconnected services. The main components of the legacy architecture were:

- **Vue 3 SPA Frontend** responsible for the user interface and interaction layer.
- **Node.js/Express Backend API** implementing business logic, workflow orchestration, and integrations with external services.
- **Redis** used for session management and temporary application state.

- **Docker Compose** used to orchestrate application containers on DigitalOcean Droplets.
- **Traefik Reverse Proxy** providing HTTPS termination and request routing.
- **Airtable** acting as the primary operational datastore for users, organizations, sites, simulations, permissions, and role assignments.
- **S3-Compatible Object Storage** used for simulation files and asset storage.
- **GitBook** used as the platform documentation system.
- **Slack and Customer.io Integrations** supporting notifications and operational workflows.
- **DigitalOcean Droplets** providing the virtual-machine-based infrastructure hosting the application stack.

From an infrastructure perspective, the architecture followed a virtual-machine-centric model. Application services were deployed as long-running containers, with scaling, upgrades, deployment management, and operational maintenance handled directly by the engineering team. While this approach was sufficient during the initial stages of the platform, it increasingly exposed limitations related to scalability, operational governance, fault tolerance, and infrastructure reproducibility.

Although the stack enabled fast iteration and reduced initial infrastructure complexity during early product development, it combined self-managed virtual-machine infrastructure with multiple external service dependencies.

3.2 Airtable as Primary Data Storage

As shown in the component overview above, Airtable was more than a persistence layer: it acted simultaneously as operational datastore, workflow-management layer, and integration hub for several internal processes.

Airtable is a cloud-based platform that combines spreadsheet-like interfaces with lightweight database capabilities. Data is organized into bases, tables, records, and fields, allowing users to manage structured information through a graphical and

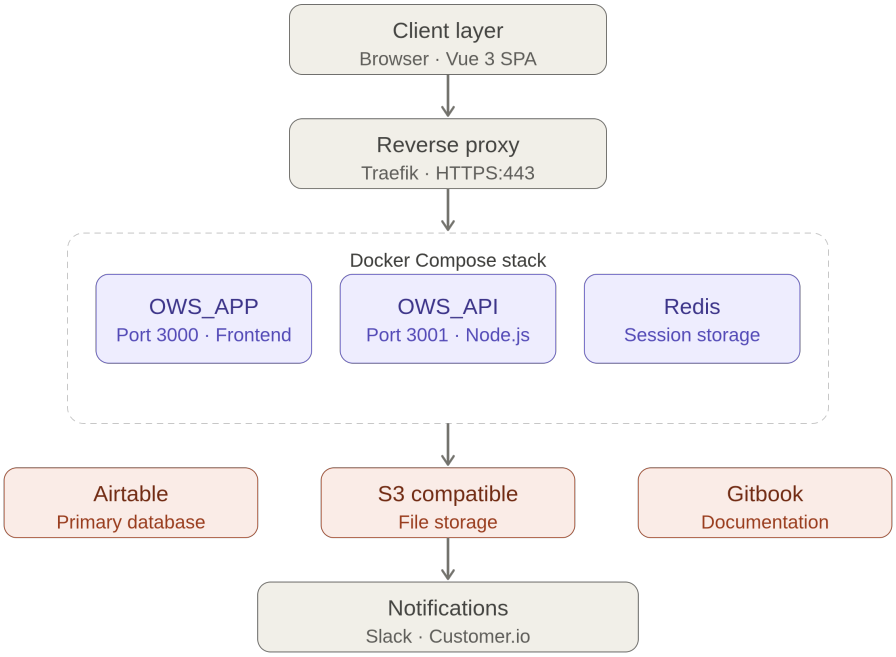

Figure 3.1. Outsight Cloud baseline architecture before the migration.

collaborative environment. Airtable also provides features such as views, forms, automations, integrations, and API access, making it particularly suitable for rapid prototyping and operational workflow management [5].

In Outsight Cloud, these capabilities were exercised through API access, operational forms, and integrations with external systems such as Slack-based notification workflows. During the early phases of the project, the accessibility of Airtable for non-technical stakeholders accelerated development and simplified collaboration between engineering and operations teams.

3.3 Limitations of Airtable

As Outsight Cloud became more complex, the limitations of using Airtable as primary backend infrastructure became increasingly evident. The issues affected not only data consistency and backend maintainability, but also operational workflows, scalability, and long-term architectural evolution.

3.3.1 Data Modeling and Consistency

Relationships between entities such as organizations, sites, simulations, and permissions were represented through linked records and lookup fields rather than through explicit relational constraints.

As a consequence, consistency guarantees depended heavily on application-side logic instead of database-level enforcement mechanisms. Backend services frequently needed to normalize and reinterpret Airtable responses in order to maintain coherent internal representations.

In several cases, lookup fields exposed heterogeneous data structures depending on the access context. The same logical information could appear either as scalar values or arrays, increasing backend complexity and complicating synchronization and mapping logic.

This absence of strict schema enforcement progressively increased technical debt and reduced maintainability as the number of entities and relationships expanded.

3.3.2 Limited Relational Capabilities

Although Airtable supports basic linked-record relationships, it does not provide the full relational capabilities of SQL database systems such as PostgreSQL.

Advanced joins, indexing strategies, transactional consistency, complex filtering operations, and optimized querying mechanisms were significantly more limited. As business logic became more sophisticated, these limitations increasingly affected backend implementation complexity and query efficiency.

The platform therefore required growing amounts of application-side logic to compensate for missing relational features that would normally be handled directly by a relational DBMS.

3.3.3 Operational Inefficiency

Several operational workflows relied on manual interactions through Airtable forms and operational interfaces.

User onboarding and account-management procedures, for example, often required manual intervention from internal teams. This approach slowed operational processes, increased dependency on human intervention, and raised the probability of operational errors.

As the number of users, organizations, and managed simulations increased, these manual procedures became progressively harder to scale efficiently.

3.3.4 Scalability, Coupling, and Cost

Over time, Airtable evolved from a simple operational datastore into a critical architectural dependency tightly coupled with application workflows and operational procedures.

Business logic progressively became dependent on Airtable-specific concepts and APIs, reducing architectural flexibility and complicating future system evolution. This dependency concentration increased technical debt and limited the maintainability of the platform.

The economic impact also became significant. Recurring operational costs associated with Airtable usage created additional pressure to adopt a more scalable

and sustainable backend architecture, without depending on a third-party platform whose cost structure was difficult to control as usage increased.

3.4 Motivation for Migration to PostgreSQL

The migration from Airtable to PostgreSQL was initiated to address the architectural, operational, and economic limitations identified in the previous sections.

The objective was not simply to replace one storage technology with another, but to progressively transition from a semi-structured operational backend toward a maintainable and scalable relational architecture capable of supporting long-term platform evolution.

PostgreSQL introduced several advantages compared to the previous data-management model. Explicit relational schemas based on primary and foreign keys enabled stronger consistency guarantees and improved integrity enforcement. Advanced querying capabilities, transactional support, and indexing mechanisms also simplified backend implementation and improved data-management efficiency.

From an architectural perspective, the migration reduced dependence on Airtable-specific structures and allowed the platform to regain ownership of its data model and persistence logic. This improved maintainability and simplified future evolution of backend services.

For these reasons, PostgreSQL was identified as the target relational datastore for the modernization effort. How that transition was executed in practice is described in Chapter 4.

Chapter 4

Migration Methodology and Decision Framework

This chapter presents the migration methodology adopted for the modernization of Outsight Cloud and explains the decision process that guided the selection of the target architecture and migration strategy. The focus is not limited to the technologies adopted, but also includes the technical, operational, and economic considerations that influenced the migration process.

4.1 Migration Goals and Success Criteria

Before defining the target architecture, a set of migration goals was identified in order to establish a structured decision framework and reduce solution bias during implementation activities.

The first objective was to avoid disruptive replacement strategies or prolonged service interruptions in production environments. This requirement translated into concrete success criteria: staged rollout capability, rollback readiness, validation of critical workflows before cutover, and separation between development and production migration activities.

Another major objective was the reduction of architectural coupling to legacy dependencies, particularly those related to Airtable-based operational workflows and platform-specific integrations. The migration also aimed to improve scalability, maintainability, and long-term architectural evolvability by adopting managed cloud

services and reproducible infrastructure-management practices.

Particular attention was also given to deployment reproducibility and environment governance. The migration strategy needed to support controlled infrastructure evolution across development and production environments while reducing manual operational activities.

In addition to technical goals, the migration process required explicit evaluation of operational sustainability and recurring infrastructure costs. Architectural decisions therefore had to balance technical quality, implementation complexity, operational overhead, and long-term maintainability.

From these objectives, several success criteria were derived. The migration process needed to support validation of critical application flows during the data-layer transition outlined in Chapter 3, provide traceable infrastructure definitions across environments, reduce dependency concentration around legacy services, and enable migration verification through tests, scripts, and CI-supported validation workflows.

4.2 Candidate Solutions Considered

The migration design process considered multiple alternatives for each architectural dimension, including infrastructure management, cloud-platform selection, data-layer evolution, and identity-management modernization.

Rather than adopting a purely technology-driven approach, the evaluation focused on identifying solutions capable of balancing migration risk, production stability, maintainability, and implementation effort.

4.2.1 Infrastructure and Runtime Options

Several infrastructure strategies were considered during the planning phase of the migration.

The first option consisted of preserving the existing deployment model while progressively hardening operational workflows and infrastructure management practices. This approach minimized short-term migration complexity but did not adequately address scalability limitations, operational governance issues, or long-term maintainability concerns.

A second option involved migrating toward managed cloud services combined with Infrastructure as Code practices. This strategy introduced a more structured cloud-native operational model while preserving incremental migration capabilities and reducing infrastructure-management overhead.

A third possibility was the adoption of a fully orchestration-oriented redesign from the beginning, for example through Kubernetes-centered infrastructure. Although this approach provided strong scalability and orchestration capabilities, it significantly increased migration complexity and operational overhead during the early transition phases.

4.2.2 Container Orchestration: Kubernetes vs ECS Fargate

During the infrastructure design phase, Kubernetes-based solutions were evaluated as a potential runtime platform for containerized workloads. Kubernetes is a portable, extensible open source platform for managing containerized workloads and services through declarative configuration and automation [6]. In production environments, it provides a framework to run distributed systems resiliently by handling scaling, failover, service discovery, load balancing, and controlled rollouts [6].

However, adopting Kubernetes would have introduced additional operational complexity that was not justified by the requirements of Outsight Cloud at the time of the migration. Managing Kubernetes clusters requires expertise in cluster administration, networking, upgrades, monitoring, capacity planning, and security governance. These responsibilities would have increased both implementation effort and long-term operational overhead.

This evaluation is consistent with Newman's caution against premature platform complexity: small service landscapes do not necessarily require Kubernetes-level orchestration, especially when the operational cost of cluster management would distract from migration goals [1]. For smaller-scale migrations, simpler managed container platforms can provide a better balance between deployment flexibility and operational effort.

In contrast, Amazon ECS Fargate provides a managed container execution environment that eliminates the need to provision and maintain worker nodes. The service offers native integration with other AWS components, including Application Load Balancer, CloudWatch, IAM, and Auto Scaling, while preserving support for

Docker-based workloads already used by the existing platform.

From a migration perspective, ECS Fargate enabled a smoother transition from the legacy Docker Compose deployment model because applications could continue to be packaged as standard containers without requiring a complete redesign of the runtime environment.

Given the project objectives (incremental migration, reduced operational burden, faster adoption, and maintainability), ECS Fargate provided the most appropriate balance between scalability and operational simplicity. While Kubernetes remains a viable option for future platform evolution, its additional complexity was not justified by the scale and requirements of the current migration effort.

Given the project constraints and the need for controlled incremental evolution, the migration strategy favored managed cloud services combined with reproducible infrastructure provisioning practices, with ECS Fargate selected as the container runtime platform.

4.2.3 Alternative Cloud Platforms Considered

During the migration design phase, multiple cloud platform alternatives were evaluated before selecting the final target environment. The comparison focused on scalability, availability of managed services, Infrastructure as Code support, ecosystem maturity, operational complexity, and compatibility with the existing technical baseline.

The main alternatives considered were Amazon Web Services (AWS), Microsoft Azure, Google Cloud Platform (GCP), continuation on the existing DigitalOcean infrastructure, and self-managed on-premises infrastructure.

Amazon Web Services (AWS). AWS was considered the strongest candidate due to the maturity of its managed-service ecosystem and its compatibility with the migration objectives of Outsight Cloud.

Services such as ECS Fargate, RDS PostgreSQL, S3, Cognito, SES, and CloudFront provided native support for the architectural model targeted by the migration strategy. In addition, AWS offered strong Terraform integration, mature DevOps tooling, and operational scalability aligned with the company's broader cloudification strategy.

The existing operational knowledge inside the company was also already partially

aligned with AWS-based workflows, reducing migration overhead and simplifying infrastructure adoption.

Microsoft Azure. Microsoft Azure was also evaluated as a potential target environment. Azure provides a broad enterprise ecosystem with managed services covering compute, storage, networking, databases, and DevOps workflows.

Equivalent services for most components used in this project were available, including Azure Virtual Machines, Azure Kubernetes Service (AKS), Azure SQL Database, and Azure Blob Storage.

Although Azure presented strong enterprise integration capabilities, especially in Microsoft-oriented ecosystems, and was considered stronger from a governance perspective, it introduced additional migration complexity because the existing operational direction and cloudification path were already more closely aligned with AWS-based infrastructure models.

Google Cloud Platform (GCP). Google Cloud Platform was considered primarily because of its strengths in containerized workloads and Kubernetes-native operations.

Relevant services included Google Kubernetes Engine (GKE), Cloud SQL, Compute Engine, and Cloud Storage. GCP also provides strong capabilities in analytics and large-scale distributed infrastructure management.

However, compared to AWS, GCP offered less alignment with the operational and architectural direction already established inside the project. For this reason, AWS was considered a more suitable choice for the migration requirements of Outsight Cloud.

Continuation on DigitalOcean. Another alternative consisted of preserving the existing infrastructure provider and progressively improving the deployment model without changing cloud platform. This approach would have minimized migration effort in the short term and leveraged existing operational knowledge. However, it would not have provided the same level of managed-service integration, infrastructure automation, and cloud-native capabilities offered by AWS. As a consequence, it was considered less suitable for supporting the long-term modernization objectives of the platform.

On-Premises Infrastructure. Another possible strategy consisted of continuing with self-managed infrastructure hosted on virtual machines or on-premises-style

environments.

Although this approach offered direct control over infrastructure resources and configurations, it significantly increased operational responsibilities related to maintenance, monitoring, backup management, scalability planning, high availability, and infrastructure security.

This option conflicted with one of the primary migration goals: reducing operational burden through managed cloud services and automation-oriented infrastructure governance.

Table 4.1. Comparative assessment of cloud provider alternatives for Outsight Cloud migration

Criterion	**AWS**	**Azure**	**GCP**	**On-prem**
Managed services fit	High	Medium	Medium	Low
IaC and automation	High	Medium	Medium	Low
Operational complexity	Low to medium	Medium	Medium	High
Ecosystem and maturity	Very high	High	High	Variable
Migration continuity with current baseline	High	Medium-low	Medium-low	Low

4.2.4 Data-Layer Migration Options

Given the limitations of the legacy datastore identified in Chapter 3, the evaluation focused on *how* to reach the target relational model, not on whether to adopt PostgreSQL.

A first possibility was an immediate cutover strategy. Although this approach simplified long-term architecture, it introduced high migration risk and limited rollback flexibility.

A second option involved maintaining long-term dual-backend operation without defining a clear migration endpoint. While operationally safer in the short term, this approach risked increasing architectural complexity and prolonging dependency concentration around legacy systems.

The selected approach instead relied on progressive validation mechanisms and controlled switchover procedures. This model allowed data-transfer correctness, API behavior, and rollback readiness to be verified before the final transition to PostgreSQL.

4.2.5 Identity Evolution Options

The migration process also required evaluating alternative approaches for identity and authentication modernization.

One option consisted of preserving existing authentication contracts and postponing IAM modernization until after infrastructure migration. While simpler initially, this strategy risked extending architectural coupling to legacy authentication assumptions.

A second possibility involved introducing standards-based IAM mechanisms progressively while preserving compatibility with existing services and workflows. This approach enabled gradual adoption of modern authentication practices while reducing migration disruption.

A third option consisted of a complete redesign of authentication and authorization mechanisms from the beginning of the migration project. Although architecturally cleaner, this approach introduced excessive migration complexity and operational risk during early transition stages.

For these reasons, the migration strategy favored progressive IAM modernization based on compatibility-preserving authentication changes and staged adoption of standards-based flows.

4.3 Technical and Economic Evaluation Criteria

Each candidate solution was evaluated against a common set of technical and operational criteria in order to support structured architectural decision-making.

4.3.1 Technical Criteria

Technical evaluation focused on aspects directly related to migration feasibility, production stability, and long-term maintainability.

Particular importance was given to backward compatibility with existing workflows and service behaviors, because API contracts, authentication assumptions, and data-access paths had to remain predictable during staged changes. Security considerations were also central, especially regarding readiness for standards-based IAM integration and cloud-native infrastructure governance.

Deployment reproducibility represented another key criterion. Candidate solutions needed to support versioned infrastructure definitions, automated provisioning workflows, and consistent environment management across development and production stages.

Migration testability was also considered essential. The selected architecture needed to support incremental validation workflows, rollback readiness, and automated verification practices capable of reducing migration risk.

Finally, operational observability and troubleshooting support were evaluated in order to ensure visibility during rollout phases.

4.3.2 Economic and Operational Criteria

In addition to technical aspects, the evaluation process also considered operational sustainability and recurring infrastructure costs.

Particular attention was given to reducing operational overhead associated with infrastructure maintenance and manual workflows. Candidate solutions were also evaluated according to their ability to reduce dependency concentration around high-cost external services and simplify long-term operational governance.

Maintainability, scalability, and automation capabilities were therefore analyzed together with recurring infrastructure costs and migration effort distribution.

4.4 Selected Strategy and Rationale

The selected strategy was an incremental migration centered on managed cloud services, Infrastructure as Code practices, and independently verifiable migration steps.

The migration involved replacing the legacy infrastructure with a cloud-native architecture built on AWS managed services. The selected strategy prioritized progressive modernization of infrastructure, deployment workflows, and data-management

components through changes that could be validated separately.

AWS-managed services were adopted for runtime execution, routing, storage, authentication, and observability, with ECS Fargate selected over Kubernetes as the container runtime platform (see Section 4.2.2). Terraform became the primary infrastructure control layer responsible for reproducible provisioning and environment governance.

For the data layer, the strategy applied the controlled transition model defined in Section 4.2.4, building on the PostgreSQL target identified in Chapter 3.

AWS was selected because it provided the most complete alignment with the technical and operational requirements of the project. In particular, the platform offered mature Terraform integration, strong scalability characteristics, managed services aligned with the target architecture (including RDS PostgreSQL), and compatibility with the broader cloudification strategy already adopted by the company.

AWS was also considered highly developer-friendly, with broad customizability enabled by Lambda-based logic and service integrations that fit the operating model of a SaaS company requiring implementation flexibility.

The main trade-off acknowledged during the evaluation is that AWS can become expensive under specific scaling and usage conditions. However, for this project, the balance between service quality, customization freedom, and migration continuity remained favorable to AWS.

The selected approach balanced migration risk, implementation effort, operational sustainability, and long-term maintainability. Its rationale was to make each architectural change independently verifiable, so that modernization could proceed through controlled increments rather than a single high-risk replacement event.

4.5 Risk Management and Coexistence Plan

Risk management was integrated directly into the migration strategy rather than treated as a separate operational activity.

Migration steps were designed to remain independently verifiable through validation workflows, data-consistency checks, and coexistence-oriented testing mechanisms. Development and production environments were managed separately in order to reduce operational risk during experimentation and rollout phases.

Special attention was given to dual-backend coexistence during data-layer migration stages. Validation workflows were introduced to detect behavioral divergence early and support controlled switchover decisions.

Data replication and validation artifacts were also designed to provide measurable evidence regarding migration progress and consistency verification.

As a consequence, coexistence was treated not as an informal transition phase, but as a structured engineering stage supported by automation, validation, and reproducible infrastructure management practices.

This approach reflects Morris's principle of reducing the scope of each change as a fundamental risk management practice: smaller increments limit the blast radius of any failure and make it easier to detect, isolate, and correct problems before they propagate [2].

Chapter 5

Target Architecture and Solution Design (To-Be)

This chapter presents the target architecture resulting from the decision framework in Chapter 4. The baseline as-is system is described in Chapter 3; here the focus is on the migration destination designed for incremental adoption.

5.1 Architecture Overview

The migration of Outsight Cloud from the legacy deployment model to AWS required the definition of a cloud-native architecture able to improve scalability, maintainability, and service integration while preserving the original application workflow.

The selected architecture follows a layered model with four levels:

- client and delivery layer;
- load balancing and routing layer;
- application layer;
- managed services layer.

The resulting left-to-right request flow starts from user interaction through the browser, continues through CloudFront and an Application Load Balancer, reaches frontend and backend services on ECS Fargate, and integrates with managed services including PostgreSQL, Redis, Amazon S3, Cognito, SES, and SNS (Figure 5.1).

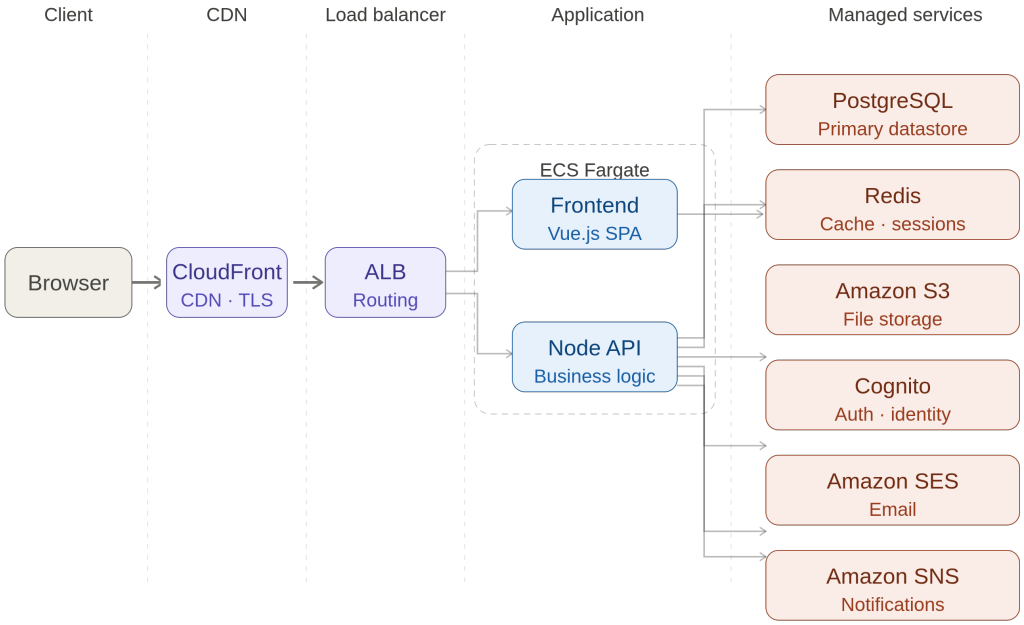

Figure 5.1. Final AWS architecture adopted for Outsight Cloud.

5.2 Client and Delivery Layer

5.2.1 Browser

The browser is the user entry point to Outsight Cloud. Through the frontend interface, users perform simulation visualization, organization and site management, document access, and file upload/download operations. All interactions occur over HTTPS.

5.2.2 Amazon CloudFront

Amazon CloudFront is used as the content delivery layer between users and the internal application infrastructure. Its role is to reduce latency, cache static assets, accelerate delivery, improve availability, and reduce backend load. This is particularly relevant for frontend static resources generated by the Vue application.

5.3 Load Balancing Layer

5.3.1 Application Load Balancer

The Application Load Balancer (ALB) is the central routing component. It receives requests forwarded by CloudFront and distributes traffic across services running on ECS Fargate. It provides request routing, health checks, HTTPS termination, and traffic distribution, enabling independent scaling and improved fault tolerance.

5.4 Application Layer

5.4.1 ECS Fargate Runtime

The application layer is deployed on Amazon ECS Fargate with a serverless container execution model. Compared to the previous self-managed deployment model, Fargate removes the need to provision, maintain, patch, and scale virtual machines manually.

This runtime choice follows the evaluation described in Chapter 4: among the container orchestration alternatives considered, ECS Fargate offered the best fit for

an incremental migration from the legacy baseline described in Chapter 3. Kubernetes was not selected at this stage because the associated cluster-management overhead (covering networking, upgrades, monitoring, capacity planning, and security governance) was not justified by the scale and operational requirements of Outsight Cloud during the migration.

5.4.2 Frontend Service (Vue)

The frontend service is implemented in Vue.js and is responsible for rendering the user interface, handling user interactions, communicating with backend APIs, and managing authentication-related flows. Deploying it as a dedicated container enables independent release and scaling behavior.

5.4.3 Backend Service (Node.js API)

The backend service is implemented in Node.js and exposes the core APIs used by Outsight Cloud. It orchestrates business logic related to organizations, sites, simulations, authentication, file operations, notifications, and integrations with AWS managed services.

5.5 Managed Services Layer

5.5.1 PostgreSQL

PostgreSQL replaced Airtable as primary data storage. The migration objective was to move from a semi-structured operational backend toward a relational model with stronger consistency and integrity guarantees. Core entities such as users, organizations, sites, and simulations are persisted in PostgreSQL.

5.5.2 Redis

Redis is used as in-memory cache and session storage. It reduces repeated accesses to persistent storage and improves response time for frequently accessed information and temporary state management.

5.5.3 Amazon S3

Amazon S3 provides object storage for simulation files, uploaded assets, documentation artifacts, and deployment-related resources. It was selected for scalability, durability, and native integration with the AWS ecosystem.

5.5.4 Amazon Cognito

Amazon Cognito is used for authentication and identity management, including user registration, login workflows, identity verification, and token handling. This reduces custom authentication logic in the application layer.

5.5.5 Amazon SES

Amazon SES is used for outgoing email communication such as invitations, account-related messages, notifications, and operational communications.

5.5.6 Amazon SNS

Amazon SNS is adopted for asynchronous event-driven notifications. It supports decoupled communication patterns for alerts, internal notifications, workflow events, and service integration triggers.

5.6 Network and Security Topology

The AWS deployment of Outsight Cloud was designed around a simplified cloud-native networking model focused on incremental migration, operational simplicity, and controlled exposure of public services.

The infrastructure operates inside an AWS Virtual Private Cloud (VPC) and relies on multiple public subnets distributed across different Availability Zones in order to improve service availability and fault tolerance. During the migration phase, the selected architecture prioritized deployment simplicity and rapid operational adoption over complex network segmentation strategies.

Application services running on Amazon ECS Fargate are deployed with public outbound connectivity enabled. This configuration simplifies communication with

externally managed services and reduces infrastructure overhead by avoiding additional networking components during the early migration stages.

Despite the use of public subnets, direct external access to application containers is restricted through dedicated Security Groups. Network exposure responsibilities are separated between the Application Load Balancer (ALB) and ECS services: the ALB accepts inbound HTTP and HTTPS traffic from external clients; ECS services accept inbound traffic only from the ALB; application containers are therefore not directly exposed to the Internet.

The Application Load Balancer acts as the primary public entry point for the platform and is responsible for HTTPS termination, request routing, health checks, and traffic distribution across frontend and backend services.

Transport Layer Security (TLS) is managed through AWS Certificate Manager (ACM), enabling centralized certificate lifecycle management and secure HTTPS communication across deployment environments.

The architecture also separates application traffic from static asset delivery. Simulation files and large downloadable resources are stored in private Amazon S3 buckets and distributed through Amazon CloudFront. The CDN layer operates independently from the application load balancer and is dedicated exclusively to content delivery and asset acceleration.

To reduce direct exposure of storage resources, access to S3 objects is restricted through CloudFront Origin Access Control (OAC), allowing content retrieval only through the CDN distribution layer.

Overall, the network topology reflects a pragmatic migration-oriented design that balances security, operational simplicity, and incremental cloud-native adoption while preserving the possibility of future evolution toward more isolated and fully private network architectures.

5.7 Architectural Benefits

Compared to the legacy architecture presented in Chapter 3, the target AWS-based design introduces several architectural improvements that directly address the migration drivers identified in Chapter 1 and the decision criteria discussed in Chapter 4.

First, the architecture provides a clearer separation of concerns between the presentation layer, application services, data storage, and supporting cloud services. This separation reduces coupling between components and simplifies future evolution of individual subsystems without requiring extensive modifications across the platform.

Second, the adoption of AWS managed services significantly reduces operational responsibility. Functions previously handled through self-managed infrastructure, such as load balancing, TLS certificate management, storage scalability, and database availability, are delegated to managed AWS services. This allows engineering effort to focus on application functionality rather than infrastructure maintenance.

The architecture also improves scalability characteristics. Frontend and backend services are deployed independently through ECS Fargate, enabling each component to scale according to its own workload profile. Similarly, managed storage and database services can evolve independently from application deployments, providing greater flexibility as platform demand increases.

From a maintainability perspective, the introduction of Infrastructure as Code through Terraform establishes a reproducible and version-controlled infrastructure model. Infrastructure definitions become reviewable artifacts that can be tracked, validated, and evolved using the same engineering practices applied to application code. This reduces configuration drift and improves consistency across environments.

The migration from Airtable to PostgreSQL further strengthens architectural ownership of the data layer. Explicit relational schemas, database-level constraints, and native query capabilities reduce application-side complexity while improving consistency guarantees and long-term maintainability.

Finally, the target architecture improves operational resilience. Environment separation, automated provisioning, managed services, and the validation mechanisms described in Chapter 6 provide a stronger foundation for controlled deployments, future migrations, and incremental platform evolution.

Overall, the final AWS architecture is not simply a replacement for the previous hosting environment. Rather, it establishes a cloud-native operational model designed to improve scalability, maintainability, operational efficiency, and long-term

sustainability while supporting the future growth of Outsight Cloud.

Chapter 6

Migration Implementation

This chapter describes the implementation foundations that were already established to execute the migration strategy in a controlled and verifiable way.

6.1 Incremental Execution Plan

The migration was implemented incrementally, with explicit separation between foundation work and full functional cutover. The implementation sequence followed four tracks:

- establish cloud infrastructure foundations on AWS;
- formalize reproducible provisioning with Terraform;
- introduce data migration tooling from Airtable to PostgreSQL;
- implement validation pipelines to compare migrated data, application behavior, and deployment readiness before cutover.

This sequencing made the implementation plan technically traceable: infrastructure readiness, provisioning reproducibility, data-transfer correctness, and application validation could be evaluated as separate workstreams before irreversible system decisions.

The infrastructure migration was intentionally performed before the complete application and data cutover. This approach allowed AWS environments, networking

components, load balancing, container execution services, and deployment workflows to be validated independently from the Airtable-to-PostgreSQL migration process.

Figure 6.1 summarizes this execution model by showing how the migration moved from the legacy baseline toward the target AWS cloud-native architecture through separate infrastructure, deployment, data-migration, and validation tracks.

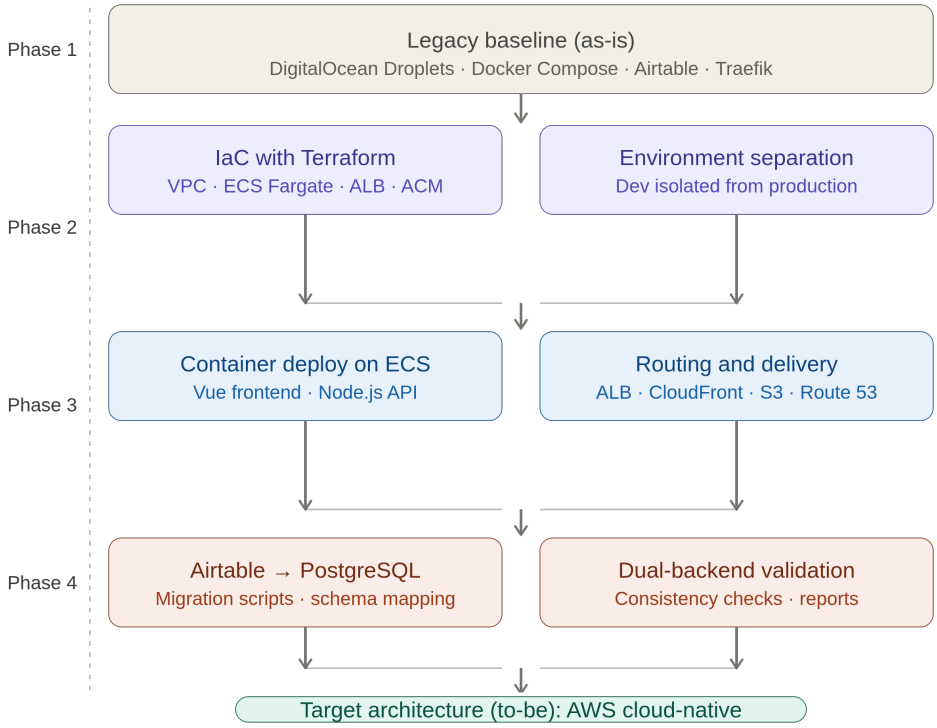

Figure 6.1. Incremental execution flow from the legacy baseline to the target AWS cloud-native architecture

6.2 Infrastructure as Code and Environment Separation

6.2.1 Infrastructure as Code (IaC)

Infrastructure as Code (IaC) is a software engineering approach in which infrastructure is provisioned and managed through code instead of manual configuration procedures [7]. The rationale comes from DevOps principles: infrastructure should be treated with the same rigor applied to application code, including versioning, reviewability, traceability, and repeatability.

Under the IaC paradigm, infrastructure definitions are declarative artifacts stored in version control systems. This replaces fragile practices based on manual console operations, ad-hoc scripts, or outdated runbooks. As highlighted by AWS guidance, declarative infrastructure improves consistency and reliability in environment creation and updates [7].

Morris identifies three mutually reinforcing core practices for IaC: defining everything as code, continuously testing and delivering all work in progress, and building small loosely coupled pieces [2]. Code defined as declarative artifacts is reusable, consistent, and transparent: everyone can see how infrastructure is built, changes are auditable, and environments can be recreated deterministically.

A key IaC characteristic is that engineers define the desired final state, while automation tooling computes and executes the required operations. This model provides:

- **Consistency:** reproducible environments across development, testing, and production.
- **Version control:** auditable change history for infrastructure evolution.
- **Automation:** integration with CI/CD-driven deployment processes.
- **Reproducibility:** deterministic creation and update workflows.
- **Maintainability:** reusable infrastructure definitions that evolve over time.

In the Outsight Cloud migration, IaC was adopted to automate AWS provisioning and reduce manual intervention during environment setup and updates.

6.2.2 Terraform-Driven Provisioning

Terraform is an Infrastructure as Code (IaC) tool developed by HashiCorp that enables provisioning and lifecycle management of infrastructure through declarative configuration files [8]. Instead of manually creating resources through cloud consoles, infrastructure is defined as code in human-readable files (HCL), which can be versioned, reviewed, reused, and integrated into software delivery workflows.

Terraform interacts with platforms through providers, i.e., plugins that map Terraform resources to external APIs. In this migration, the AWS provider was used to manage cloud resources including ECS Fargate services, Application Load Balancers, Route 53 DNS records, ACM certificates, VPC networking components, S3 storage, and observability-related infrastructure.

Listing 6.1. Simplified ECS Fargate service definition

```
resource "aws_ecs_service" "backend" {
  name = "outsight-backend"
  cluster = aws_ecs_cluster.main.id
  task_definition = aws_ecs_task_definition.backend.arn

  desired_count = 1
  launch_type = "FARGATE"

  network_configuration {
    subnets = aws_subnet.public[*].id
    security_groups = [aws_security_group.ecs.id]
    assign_public_ip = true
  }
}
```

Listing 6.1 shows a simplified Terraform resource definition used to deploy a backend container as an ECS Fargate service. Through declarative configuration files, containerized workloads previously orchestrated through Docker Compose could be provisioned reproducibly in AWS while preserving the same packaging model.

6.2.3 Terraform Workflow: Write, Plan, Apply

The Terraform workflow follows three phases [8]:

- **Write:** define the target infrastructure state declaratively in configuration files.
- **Plan:** generate an execution plan that compares desired state and current state, showing which resources will be created, updated, or removed.
- **Apply:** execute the planned operations while respecting resource dependencies through an internal dependency graph.

This workflow was especially useful for migration safety, because planned changes could be reviewed before execution and applied incrementally.

6.2.4 State Management and Team Collaboration

Terraform keeps a state representation of real infrastructure (`terraform.tfstate`) to compute incremental changes instead of recreating complete environments [8]. In practice, state management enabled drift detection and made environment updates predictable across migration iterations.

For collaborative operations, shared state and controlled execution are essential. This improves consistency across development stages and reduces configuration drift during parallel workstreams.

6.2.5 Rationale for Adopting Terraform

Terraform was selected because it aligned with the migration objectives:

- **Standardization:** infrastructure definitions became versioned artifacts.
- **Automation:** provisioning and updates were integrated into CI/CD-oriented processes.
- **Consistency:** environments were reproducible across development and production contexts.
- **Scalability:** infrastructure changes could be applied incrementally.
- **Maintainability:** modular and declarative definitions reduced manual intervention.

Within the Outsight Cloud project, this translated into a reproducible AWS infrastructure baseline supporting the broader cloudification strategy and reducing operational risk during migration.

6.2.6 Environment Governance

A dedicated effort was made to separate development and production concerns. This included environment-specific configuration handling and controlled evolution of shared resources to avoid production side effects during migration experimentation.

This separation was particularly important during the migration from the legacy infrastructure because it allowed AWS environments to be validated independently before production workloads were progressively redirected to the new platform.

6.3 Infrastructure Migration Execution

While Chapters 3 and 5 describe respectively the legacy platform and the target AWS design, this section documents how the transition to AWS was executed in practice. The objective was to replace the previous Docker Compose and Traefik deployment model with a reproducible cloud-native foundation without forcing a simultaneous application and data cutover.

6.3.1 Target Environment Definition and Terraform Provisioning

The first implementation step consisted of translating the target architecture into an Infrastructure as Code baseline using Terraform. Rather than manually configuring AWS resources through the console, the migration team defined a declarative stack covering the core components required to host Outsight Cloud in AWS.

The initial Terraform implementation provisioned the main building blocks of the target platform, including ECS Fargate services for the frontend and backend containers, an Application Load Balancer for public routing, Route 53 records for DNS management, ACM certificates for HTTPS, CloudWatch log groups for centralized logging, and S3 together with CloudFront for simulation file storage and

delivery. This stack established the first operational AWS environment capable of replacing the legacy hosting model.

6.3.2 Routing, Application Adaptation, and Container Deployment

A key difference between the legacy and target deployment models concerned request routing. In the previous environment, Traefik acted as reverse proxy and HTTPS entry point for containerized services. In the AWS architecture, routing responsibilities were transferred to the Application Load Balancer and defined through listener rules that map incoming traffic to the appropriate target groups.

The target design separated public user-interface traffic from API traffic at the load-balancer layer. Rather than depending on proxy-level path rewriting, the backend service was adapted so that its externally exposed routes aligned with the ALB routing model, while a dedicated health-check endpoint remained available for service and target-group probes. This alignment reduced implicit routing assumptions, simplified the deployment topology, and made request handling easier to validate during migration.

Container images were published through the existing private registry used in the delivery pipeline. To allow ECS tasks to pull images securely, registry credentials were supplied at runtime through AWS Secrets Manager instead of being embedded in Infrastructure as Code artifacts or Terraform state. Frontend and backend workloads were deployed as separate ECS Fargate services, preserving the container packaging model already used in the legacy platform while adapting orchestration and credential handling to AWS operational practices.

6.3.3 Storage, TLS, and Content Delivery

Simulation file storage was redesigned around a private S3 bucket fronted by CloudFront. This replaced the previous S3-compatible storage integration with an AWS-native delivery model that improved scalability and reliability while keeping the bucket non-public. Access to stored objects was restricted through CloudFront Origin Access Control, and Terraform also defined IAM permissions for upload workflows and cache invalidation.

Public HTTPS access was enabled through ACM-managed certificates integrated with the ALB and CloudFront distribution layers. Route 53 was used to manage DNS records either through an existing hosted zone or through a delegated subdomain, depending on the deployment environment. Together, these components replaced the certificate and routing responsibilities previously handled by Traefik in the legacy environment.

6.3.4 Environment Validation and Controlled Service Transition

Once the AWS environment was provisioned, validation activities focused on verifying that the new infrastructure behaved correctly before redirecting production traffic. The validation scope included ECS service startup, ALB routing between frontend and backend paths, HTTPS and DNS configuration, centralized logging, and the S3/CloudFront flow for simulation assets.

Development and production concerns were kept separated throughout this process. In particular, experimental infrastructure changes and database modernization activities were isolated in development-oriented environments so that AWS networking, deployment workflows, and service exposure could be tested without affecting the production system still running on the legacy platform.

The final transition step consisted of progressively redirecting public access from the legacy environment to the AWS platform through DNS updates managed in Route 53. This sequencing allowed the team to validate the new stack independently, compare behavior against the existing deployment, and reduce migration risk by avoiding a single disruptive cutover across infrastructure, application routing, and data storage at the same time.

6.4 Data Migration Tooling and Validation

6.4.1 Relational Foundation

The migration path from Airtable to PostgreSQL was implemented with schema and migration tooling designed to preserve compatibility constraints. The objective was not only data transfer, but controlled evolution toward explicit schema ownership.

6.4.2 Replication and Validation Scripts

Dedicated scripts were prepared to:

- replicate data from Airtable into PostgreSQL;
- validate consistency through machine-readable reports;
- support migration checkpoints before wider rollout.

These artifacts transform data migration from a one-time operation into a repeatable verification workflow.

6.5 Implementation Scope and Current Status

At the current stage, the implemented work establishes the migration backbone:

- AWS infrastructure provisioning and controlled transition from the legacy baseline;
- environment governance and Terraform-driven deployment foundations;
- data replication and validation tooling.

This scope is sufficient to evaluate migration feasibility and design quality, while leaving room for further functional expansion in subsequent project phases.

Chapter 7

Evaluation and Discussion

7.1 Benchmark Methodology and Demand Profile

7.1.1 Measurement Scope

7.1.2 Workload and Demand Definition

7.1.3 As-Is vs To-Be Benchmark Setup

7.2 Cost and TCO Analysis

7.2.1 Current Architecture Costs (As-Is)

7.2.2 Target Architecture Costs (To-Be)

7.2.3 Total Cost of Ownership (TCO)

7.3 Latency and Performance Analysis

7.3.1 As-Is Latency Measurements

7.3.2 To-Be Latency Measurements

7.3.3 Comparative Results

7.4 Operational Cost and Efficiency

7.4.1 As-Is Operational Effort

7.4.2 To-Be Operational Effort

7.4.3 Operational Cost Comparison

Chapter 8

Conclusions and Future Work

This thesis investigated the modernization of Outsight Cloud through the migration of a production platform from a legacy infrastructure toward a cloud-native architecture on AWS.

The original system relied on self-managed infrastructure and Docker Compose, while depending on several external services, most notably Airtable as the primary operational datastore. Although this architecture enabled rapid development and operational flexibility during the early stages of the product lifecycle, it progressively introduced limitations related to scalability, maintainability, operational governance, and dependency management.

Rather than approaching migration as a simple infrastructure relocation exercise, this work treated cloud adoption as a broader architectural transformation problem involving infrastructure modernization, data-layer evolution, deployment automation, and operational redesign. Particular attention was given to the engineering artifacts that made this transformation controllable, including Terraform definitions, data migration tooling, validation scripts, and environment-specific deployment workflows.

8.1 Key Outcomes

The first outcome of this work is the definition of a structured decision framework capable of supporting migration activities under real industrial constraints. The analysis demonstrated that successful modernization initiatives require balancing

technical quality, operational sustainability, implementation effort, and long-term maintainability.

A second contribution is the design of a target AWS architecture based on managed services, Infrastructure as Code principles, and cloud-native operational practices. The resulting architecture replaces the previous virtual-machine-based deployment model with a managed container platform built around ECS Fargate, PostgreSQL, CloudFront, Application Load Balancers, and supporting AWS services.

The work also demonstrated the value of Infrastructure as Code as a foundational migration enabler. By adopting Terraform as the infrastructure management layer, infrastructure resources became reproducible, version-controlled, auditable, and easier to evolve across development and production environments.

Another important outcome concerns the migration strategy itself. Data replication tooling, validation scripts, and consistency checks were introduced to make migration progress measurable and verifiable while reducing operational risk.

The case study also shows that technical outcomes and operational outcomes were closely connected: the transition toward an AWS-native architecture required coordinated changes across infrastructure, deployment workflows, data management, and operational governance.

Finally, the artifacts produced throughout the project (architectural evaluations, Terraform configurations, migration tooling, and validation workflows) provide reusable practices that can support similar modernization initiatives in other industrial contexts.

8.2 Future Work

Although the migration foundations have been established, several activities remain as future developments.

The first direction concerns the completion and monitoring of the full production migration process. Long-term operational metrics should be collected in order to evaluate system reliability, scalability, availability, and user experience after complete adoption of the target architecture.

A second area of future work involves the continued evolution of identity and

access management. While the migration introduced modern authentication services, additional improvements could further standardize authorization models and strengthen integration across platform components.

Another important direction is the extension of observability and operational analytics capabilities. Enhanced monitoring, centralized logging, and performance analysis could provide deeper visibility into application behavior and support data-driven operational decisions.

Future work may also investigate more advanced cloud-native capabilities, including automated scaling strategies, additional resilience mechanisms, and further optimization of infrastructure costs as platform usage grows.

Finally, a comprehensive post-migration evaluation should be performed once the platform is fully operating on AWS. Such an assessment would enable a detailed comparison between the legacy architecture and the final cloud-native solution in terms of performance, operational effort, reliability, security, and total cost of ownership.

Overall, this thesis demonstrates that legacy-to-cloud-native migration is most effective when treated as a structured engineering process supported by explicit architectural decisions, reproducible infrastructure management, continuous validation, and controlled incremental evolution. The Outsight Cloud case study shows how modernization can be grounded in concrete engineering practices: declarative infrastructure, managed runtime services, relational data modeling, automated validation, and risk-aware rollout planning.

Bibliography

[1] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. O'Reilly Media, 2021.

[2] K. Morris, *Infrastructure as Code: Dynamic Systems for the Cloud Age*, 2nd ed. O'Reilly Media, 2021.

[3] Amazon Web Services. «What is cloud migration?» Accessed: 2026-05-17. (2026), [Online]. Available: `https://aws.amazon.com/what-is/cloud-migration/`.

[4] Amazon Web Services. «What is iam?» Accessed: 2026-05-31. (2026), [Online]. Available: `https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html`.

[5] Airtable. «Introduction to airtable basics». Accessed: 2026-05-25. (2026), [Online]. Available: `https://support.airtable.com/docs/introduction-to-airtable-basics`.

[6] The Kubernetes Authors. «Overview of kubernetes». Accessed: 2026-05-30. (2026), [Online]. Available: `https://kubernetes.io/docs/concepts/overview/`.

[7] Amazon Web Services. «Infrastructure as code». Accessed: 2026-05-17. (2026), [Online]. Available: `https://docs.aws.amazon.com/whitepapers/latest/introduction-devops-aws/infrastructure-as-code.html`.

[8] HashiCorp. «What is terraform?» Accessed: 2026-05-17. (2026), [Online]. Available: `https://developer.hashicorp.com/terraform/intro`.